Evaluating range queries on interval and range encoded time series data with RUBIK

Vlad Mihaescu
École Polytechnique Fédérale de Lausanne
Lausanne, Switzerland
vlad.mihaescu@epfl.ch

ABSTRACT

An increasingly prevalent type of data that has emerged more recently because of the advent of faster computer chips, better sensors and cheaper storage is timeseries data. From stock trading applications, to smart factories, home automation devices and driverless cars, all generate huge amounts of valuable information. With this type of data, we usually only add to it as time goes on, i.e. it is append-only, and time is the main axis, which is also the reason why one can extract a plethora of new insights from the data, as it changes with time. Moreover, in the sciences, data generated by simulations is crucial for not only allowing scientists to understand the phenomena studied, but also to be able to further improve and refine the models used. Timeseries data in the sciences in particular, which is the type of data that is the focus of this report, is thus posing an interesting challenge. For most cases, if one uses traditional databases, the size of the data is too big for scientists to perform exploratory queries in real-time, and time-consuming queries are not a feasible solution.

To address this issue, a novel way of running queries on vast amounts of timeseries data in real-time has been developed, called RUBIK [6], whereby each timeseries is first discretized by binning the observation values in a pre-set number of bins, and is then further range-encoded as a two-dimensional bitmap, of time and observation value dimensions. The resulting bitmaps are then stacked into three-dimensional bitmaps, where the third dimensions represent the number of bitmaps, and then compressed, which allows threshold queries of the form "$A \leq v$" or "$A \geq v$" to be executed efficiently on the compressed representation by taking advantage of a Quadtree structure and its pruning power.

In this project, we extend RUBIK by adding the ability to perform two-sided range queries on the range-encoded bitmaps, and taking note from [2], we add a new type of encoding more well-suited for performing two-sided range queries, called interval encoding. Lastly, we test and compare the two types of encoding on both types of queries, namely one-sided range queries of the type $A \leq v$ or $A \geq v$ and two-sided range queries of the type $v1 \leq A \leq v2$, in terms of index size, query execution time and precision. We find that using the same number of bins and the same binning strategy produces indexes that are close in size, similar in the number of hits we get, but in the case of interval encoding, showing better performance overall.

1 INTRODUCTION

In the past few years, thanks to the declining costs of computer chips and increased internet connectivity, more and more devices have been connected to the internet, allowing more data to be captured. Faster chips and more accurate sensors have allowed smart home devices to continuously generate more and more data from sensor readings, self-driving cars to collect enormous amounts of data about the environment, or the retail industry to track and monitor products with such precision that same-day delivery has become possible. Where before data was usually updated, now it is easier to append it to the already existing data, and this allows new types of insights to be gained. As an example, where before a website could store the last login time, it may now store all login times, and discover patterns of access which could not be seen before. As such, we can see that common to all the previous scenarios is the fact that the time dimension is the primary axis. When new data arrives, it is always at a later time than the latest entry, which is then appended to the rest of the data. This allows for tracking changes over time, which ultimately leads to ways of predicting changes in the future, based on how it changed in the past. In order to be able to achieve this, a vast amount of data has to be stored and queried at a later time, and the problem that arises is that since its size can grow very fast, most common databases are not suitable for efficiently querying the datasets, in real time, and new methods are needed, that allow scientists to explore it fast and efficiently. In the sciences in particular, thanks to the availability of supercomputers and cheap storage options, scientists are able to perform more accurate simulations, as well as increase their size. This leads to a vast amount of timeseries data being collected, and the need for tools to efficiently sift through it is essential for enabling scientists to make new discoveries and gain new information. By analyzing time series, scientists can refine their models and make them more accurate. However, since the size of the collected data is so vast, they cannot analyze all of it in detail. However, what is fortunate is that not all of the collected data is of the same importance, and only some parts are really needed in order to obtain new insights, and it is only these small subsets that need to be queried efficiently and analyzed in detail. In this report, we are concentrating on the problem of finding these small interesting subsets of data, using one-sided and two-sided range queries, i.e. queries that are below, above, or in-between certain user-provided values. In addition, constraints on the time axis can also be placed, allowing for the queries to be executed only on a certain time segment of the data. Two encoding schemes are used in order to answer these types of queries, each being better suited to a certain query type. Typically, the way queries on time series data are being answered is by compressing the data in a lossy way and then using it as an index that approximately answers the query. In order to validate the answer, the exact time series need to be read from disk. Thus, the better the approximation is, the less time series data needs to be accessed and read from disk, resulting in a faster answer, as disk reads are orders-of-magnitude slower than reads from memory.

Other approaches consider compressing along the time axis by averaging multiple discrete time steps for example. However, doing so, means a very large number of false positives will be included, as well as false dismissals, since compressing along the time axis means that we may not be able to identify the original interval where the time series exceeds a certain threshold. This project is an extension of RUBIK, an innovative compression technique used to answer queries in a very fast and efficient way. With RUBIK, the time axis is not compressed, and only the observation values are, by the process of binning. Afterwards, range-encoded two-dimensional bitmaps are created, which are then encoded by a Quadtree structure and in the end compressed using Word-Aligned-Hybrid (WAH) compression [7]. The queries are then executed on the Quadtree structure as two-dimensional spatial range queries. This project makes the following contributions:

- It extends RUBIK by adding the ability to run two-sided range queries on the range-encoding scheme
- Adds a new encoding scheme that is theoretically better suited to answer two-sided range queries, called interval encoding, and adapts it to answer one-sided and two-sided range queries
- Performs a comparative analysis between the two schemes to determine the suitability of each one for the different types of queries

We find that the size of the index increases linearly with the size of the dataset, and empirically show that when using the same number of bins, an interval-encoded index has a shorter query execution time compared to the range-encoded index, while producing approximately the same number of hits, i.e. definite results.

2 BITMAP INDEXING OVERVIEW AND RELATED WORK

Bitmap indexes are one of the most efficient ways of making query evaluation faster on read-only or read-mostly data, especially because of their use of bitwise logical operations, which are well-supported and efficiently implemented in computer hardware. In order to create a bitmap index, three main steps are performed, namely binning, encoding and compression. Binning essentially assigns multiple attribute values to one range. The encoding scheme determines how the attribute values in the records are represented by the bitmap, meaning which bits of the bitmap are set to 1. Finally, compression reduces the size of the index by, usually, representing consecutive runs of the same bit values as a (value, length of sequence) pair.

2.1 Bitmap Index Binning

Bitmap indexes work well with low-cardinality attributes, for example the "types of car models of a car company", "gender", "cities", etc. However, scientific data does not follow the same patterns, and usually has very high-cardinality, as the data values are floating-point numbers. Thus, to be able to reduce the index size and make it manageable, as well as to be able to control the size, binning is applied, which places multiple distinct values in the same bitmap, thus reducing the number of bitmaps that will be used in the index [9]. Instead of forming the bitmap for each individual value, we form it for each bin. The disadvantage of this strategy is that now we are not sure whether the result of the query contains some false positives, i.e. values outside of the query range that may have fallen in the same bin or bins as the ones inside the query range. As an example, for a query such as "voltage between 8.1 and 16.7" , we can imagine it hits the bins between 12 and 24; the interior bins, i.e. bins 13 to 23 are definite hits and we do not need to check on disk; edge bins however, i.e. bins 12 and 24, cannot guarantee hits as some values, such as 8.0 or 7.9 may have also fallen in bin 13 but are not a valid result. Thus, for the edge bins, binning introduces the additional step of candidate checking, whereby we access the data on disk to validate the result. Even though the number of false positives may be small, they are usually spread all around the dataset, and we end up touching most of the disk pages storing the dataset which are usually large (4KB or 8KB) [9]. Ideally, one would represent each value individually so that one can answer all queries without the need to perform candidate checks on the base data, which, as mentioned previously, dominates query time. However, since the values in scientific data are usually floating-point, and too much space would be needed to represent each value individually, a very fine binning could be defined, and one could assume that the query can be no finer than the finest level of those bins. In essence, whenever a value would fall in that bin, we would assume that all the other values in the bin match the query. In terms of binning, one could use binning by either equi-width or equi-depth, as both have about the same outcome. [4]

2.2 Bitmap Index Encoding

Encoding consists in transforming the binned values into bitmaps. The simplest encoding scheme is equality encoding, whereby the presence of a value in a particular row is signaled by placing a "1" in the corresponding position of the bitmap. Chan and Ioannidis describe two additional encoding schemes, range [1] and interval [2] encoding, the former being suited for one-sided range queries of the type "$v \leq 54$" and the latter more suitable for two-sided range queries such as "$12 \leq v \leq 33$".

2.2.1 ***Equality Encoding.*** In its simplest form, one bitvector in the bitmap represents one attribute value, a scheme called equality encoding. In this scheme, building an index for a particular attribute A generates C bitmaps (C being the cardinality), each of size N, where N is the number of rows of the dataset. For any row, if the attribute value is present in the bitmap, the corresponding bit entry is set to "1", otherwise it is set to "0". This type of encoding is very suitable for performing equality queries of the form $A = v$, as only one bitmap needs to be accessed for the query to be answered.

2.2.2 ***Range Encoding.*** For queries of the form $A \leq v$ or $A \geq v$, known as one-sided range queries, a different encoding scheme called range encoding, described in [1], is better suited, as each bit vector represents a one-sided relation of the form $A < v$, as opposed to equality encoding, where each bit vector represents a relation of the form $A = v$. The bitmap encoding already implemented in RUBIK is range encoding. With this scheme, for each timestep, every bin above the one where the observation value falls in is represented by a 0, and everything below (and including the current one) is represented by a 1. The kth bitmap can more easily be thought of as the bitwise OR of the first k bitmaps under equality

encoding. It is because of the fact that each bitmap covers a lot of information about the domain of the attribute that we can answer a query by only needing two bitmaps. This encoding is particularly suited for answering queries of the type $A \leq v$ or $A \geq v$. For the latter type of query, we can easily see that since $A \geq v \equiv NOT(A \leq v-1)$, the answer can be found by computing $NOT(A \leq v-1)$.

2.2.3 ***Interval Encoding.*** In the interval encoding scheme, described in [8], the kth bitmap represents the bitwise OR of half of the bitmaps under equality encoding, starting with the kth bitmap. Because of this construction, under interval encoding, for N distinct values, we need $\lceil N/2 \rceil$ bitmaps, resulting in a smaller size, when compared with equality and range encoding schemes that use N bitmaps. With interval encoding any query can be answered by using just two bitmaps, just as with range encoding. The main benefits of this type of encoding are a reduction of the number of bitmaps used as well as a more efficient way of evaluating queries of the type $v1 \leq A \leq v2$ [8]. As noted by [5], no commercial system has implemented this encoding scheme ever since the encoding scheme was proposed. The reason is that compression is not efficient, resulting in a much larger index size.

2.2.4 ***Multi-Level Encoding.*** Apart from the equality, range, and interval encoding schemes for bitmap indexes, other schemes have been proposed, that, while more complex, promise better performance and a lesser number of false positives in the result set, thus requiring fewer disk accesses for result validation. As described in [4], when dealing with high-cardinality data, the main drawback of using equality encoding is the large number of bitmaps that need to be generated which makes the index size unmanageable, as well as the large number of bitmaps that need to be accessed in order to answer a range query, be it one-sided or two-sided. In the case of range encoding as well as interval encoding, high-cardinality data, assuming no pre-binning, also produces indexes that may be even larger than the base data and in addition they may be hard to compress, much more so even in the case of interval encoding. The advantage however is that we only need to access two bitmaps to answer a range query, whether it one-sided or two-sided. One of the proposed alternatives would be to use the so-called multi-level encoding, which is in essence a combination of two types of encoding. Of note are the range-equality and interval-equality encoding, whose main advantage is that with no binning, they require no candidate checks, significantly reducing the query execution time, as disk accesses dominate time spent on query execution. Using WAH compression [7], it has been shown that these two types of encodings outperform the basic bitmap indexes. A two-level interval-equality index has, as the name suggests, two levels. At the highest level, coarse, interval-encoded bins are used, and at the lowest level, equality encoded bins are used.

2.3 Bitmap Index Compression

Compression is performed on the bitmaps as a final step, so that space can be saved. A very simple compression scheme that is lossless is run-length encoding. With this scheme, consecutively repeated values are not stored individually, but rather as a data value and count pair. If the encoding scheme generates many uniform runs of data, it is a very efficient compression scheme. Word-Aligned-Hybrid compression, described in [7], is a variant of run-length encoding. A word in the WAH scheme can be either a "literal" or a "fill" word. Any type of word has a flag indicating which of the two types it is, with a "literal" word containing the actual bit sequence in uncompressed form. A "fill" word on the other hand represents how many consecutive 1s or consecutive 0s there are. The second bit stores the represented value (either "1" or "0") and the remaining bits represent the length of the sequence. Because this type of compression uses word-alignment, it operates on whole words, rather than bits or bytes, and hence is much more efficient.

3 RUBIK OVERVIEW

RUBIK takes advantage of the fact that in the simulation sciences especially, there is similarity between, as well as across, time-series data. For example, between multiple timeseries, at the same timestep t, the values may not differ significantly. Moreover, across the same timeseries, the values at $t-1, t, t+1$ may also not vary by a large margin. This is of course not the case when there are spikes in measurements, etc. By taking advantage of this similarity, RUBIK can aggregate multiple 2D bitmaps and group them together if they are the same at a coarser level, not at the individual timestep level. The level of coarseness is an important factor as it presents a trade-off between query speed and size of the index; by having a higher level of coarseness, the number of clusters decreases, which means less time is spent querying; at a lower level of coarseness we have a larger number of small clusters, which occupy more space but have a better compression ratio. We can imagine the stacked bitmaps as a 3D cube, which is then serialized on disk by splitting it in four equal quadrants, and checking whether they are *ALL* 1, such that in all three dimensions, all the bits inside are a "1", similar for the *ALL* 0 case, or *MIX*, that is they contain a mixture of 1s and 0s. Should the latter be the case, the resulting block is recursively split until it either contains *ALL* 1, *ALL* 0 or cannot be split anymore, as seen in Figure 1. At the top of Figure 1, we have the 3D cluster of bitmaps, which is then split into four quadrants along the time and observation dimensions, resulting in three uniform nodes and one mixed block. The mixed blocked is further split into four quadrants, three of which are uniform and one mixed. Since each quadrant is now one-dimensional, it cannot be split anymore, and the *MIX* block will be WAH-compressed. For the *ALL* 1 and *ALL* 0 nodes, they can be very efficiently stored since we are only storing their size and whether they are 1s or 0s, not the individual bits. In case of the *MIX* nodes, RUBIK uses the very performant WAH compression scheme to further reducing the size of the index.

Since each block has the time dimension and binned observation values dimension, with a start and end time as well as a start bin and end bin, we can then run a query on the Quadtree, by traversing the data structure and checking if the query bounds (also expressed in terms of time and bin bounds, into which the actual threshold values that we want fall) intersects a node. If that is the case, each node is then added to a list. After having found all intersecting nodes, we can then perform calculations of hits and candidates, expressed in terms of a lower and upper bound. To make execution faster, we do not rely on bitwise logical operations, especially since some nodes do not even contain bits, as is the case for the *ALL* 1

and *ALL* 0 nodes. Instead, we identify the possible cases of results, and return the correct ones. For example, if one bitmap is *ALL* 1 and the operation to be performed is a logical "OR", we simply return the number of bits that the *ALL* 1 block contains.

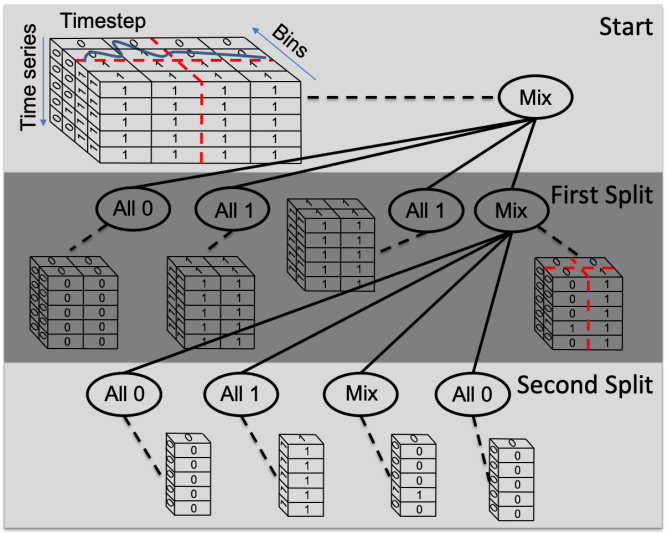


Figure 1: RUBIK Cluster Decomposition

4 RUBIK BINNING AND ENCODING

As the project is focused on scientific data, which is of very high cardinality, in the case of interval encoding it is not feasible to use the observation values as interval bounds, as we would need $C/2$ intervals, where C represents the cardinality of the base data. Thus we decided to first bin the data into a pre-determined number of bins. The first step consists in determining the bin into which the observation value falls in. As such, we reduce the cardinality C of the base data to N, where N represents the number of bins. The number of resulting intervals is then $\lfloor N/2 \rfloor$. Thus we have the intervals $\{I^0, I^1, ..., I^{\lceil N/2 \rceil}\}$, with each interval I^k covering the range of bins $[k, k + \lfloor N/2 \rfloor - 1]$. The bitmaps have the timeseries' timesteps as one dimension and the intervals $\{I^0, I^1, ..., I^{\lceil N/2 \rceil}\}$ as the second dimension. In order to fill the bitmaps, for each timestep, a bin from $[0, N]$ for the observed values is computed, and afterwards we calculate the range of intervals that contain this bin, as such: $I_{lower} = max(0, bin - \lfloor N/2 \rfloor + 1)$, $I_{upper} = min(bin, \lfloor N/2 \rfloor)$. The range between I_{upper} and I_{lower} (inclusive) is filled with 1*s* and everything outside of the range is filled with 0*s*. An important part is how to determine efficiently whether a whole three-dimensional block of the bitmap generated as stated above contains a mixture of 1s and 0s or not. Two one-dimensional vectors are kept (min and max), each the size of the number of timesteps of a timeseries, recording at each timestep t, the maximum and minimum number of the bin, across all timeseries. In the case of interval encoding, we find I_{upper} and I_{lower} with the functions described above. When checking a Quadtree node defined by a start time $t1$ and an end time $t2$, we take a slice of the minimum and maximum vectors from the start ($t1$) to the end time ($t2$). For every timestep between $t1$ and $t2$ we now look whether the bins in the min vector are all above the query start voltage and whether the bins in the max vector are all below the query end voltage. As we know that the bitmap is filled with 1s between that interval upper range and interval lower range, according to the interval encoding scheme, we can now say if a block is *ALL* 1. We perform similar checks for the *ALL* 0 case. If neither of the cases are true, we can affirm that the block is *MIX*.

input : ts: timeseries array
t: number of timesteps
N: number of bins
lower interval: first interval that covers a range that contains the bin b
upper interval: last interval that covers a range that contains the bin
output : bitmap: 2D array representation of timeseries data

$I_{lower} = max(0, bin - \lfloor N/2 \rfloor + 1)$
$I_{upper} = min(bin, \lfloor N/2 \rfloor)$

for $i = 0; i < t; i++$ **do**
  for $j = interval_{lower}; j \leq interval_{upper}; j++$ **do**
    $bitmap[i][j] = 1$;
  end
  if $interval_{lower} > 0$ **then**
    for $j = 0; j < interval_{lower}; j++ \leq interval_{upper}$ **do**
      $bitmap[i][j] = 0$;
    end
  end
  if $interval_{upper} + 1 < N$ **then**
    for $j = interval_{upper} + 1; j < N; j++$ **do**
      $bitmap[i][j] = 0$;
    end
  end
end

Algorithm 1: Creating interval-encoded bitmaps

5 RUBIK QUERY EXECUTION

Query execution is performed by constructing a two-dimensional query for the time and observation dimensions. Each query is thus defined by bounds t_{lower} and t_{upper} in the time dimension and intervals i_{lower} and $iuppper$ in the observation dimension. The two intervals are computed before constructing the query, based on Equations (1) or (2), and they represent the two intervals needed in order to answer a query based on the interval encoding scheme. For efficiency, such that two separate queries need not be constructed separately, we overload one query with four intervals, two of which are for computing the lower bound, and the other two for the upper bound. The constructed query is then executed on the Quadtree by traversing the data structure and checking whether the query bounds intersect with the nodes of the Quadtree. The nodes that do are put into a list so that later we can perform operations on the bitmaps to find the number of hits and candidates. The sets of equations (1) and (2) are used to determine which operations should

be performed on the two intervals required to answer the query for both one-sided and two-sided queries under interval encoding. The equations are for queries of the form $A \leq v$. To be able to answer queries of the form $A \geq v$ we simply negate the query as such $NOT(A \leq v)$. In the case of one-sided range queries, we use equation (1), and we denote by v the value for the query and by b the bin into which the query value falls, where $0 \leq b \leq N-1$, where N is the number of bins. For two-sided range queries of the form $v1 \leq A \leq v2$, we apply equation (2) and denote by $b1$ the bin where value $v1$ falls, and $b2$ the bin where value $v2$ falls.

We further write $m = \lfloor N/2 \rfloor - 1$

$0 \leq b \leq N-1$

$$A \leq v = \begin{cases} I^0 \wedge \overline{I^{b+1}} & \text{if } v < m \\ I^0 & \text{if } v = m \\ I^0 \vee I^{b-m} & \text{if } m < b < N-1 \end{cases} \quad (1)$$

$$v1 \leq A \leq v2 = \begin{cases} I^{v1} \wedge \overline{I^{v2+1}} & \text{if } v2 < m, \\ I^{v1} \wedge I^0 & \text{if } v = m \\ I^{v1} \wedge I^{v2-m} & \text{if } v2 < v1 + m, v1 < m \\ I^{v1} & \text{if } v2 = v1 + m, v1 < m \\ I^{v1} \vee I^{v2-m} & \text{if } v2 > v1 + m, v1 < m \\ I^{v1} \vee I^{v1+1} & \text{if } v2 = v1 + m + 1, v1 = m \\ I^{v2-m} \wedge \overline{I^{b1-m-1}} & \text{if } b1 \geq N \end{cases} \quad (2)$$

Since we are also dealing with nodes that are *ALL* 1 or *ALL* 0 and thus do not store the individual bitvectors, as well as from a performance point of view, the implementation of the operations in Equations (1) and (2) does not use actual *AND* and *OR* bitwise operations, rather it computes them based on all the cases that might arise. Equations (1) and (2) give the upper bound. To compute the lower bound we perform the Equations (1) and (2), replacing the second interval I^b with I^{b-1} for one-sided range queries and I^{b1} with I^{b1+1} and I^{b2} with I^{b2-1}. In range encoding, the lower bounds and upper bounds are computed based on the bin b where the query value falls and bin $b-1$, as described in [6].

6 ANALYSIS AND TESTING

This section consists in an empirical evaluation of the performance of the two encoding schemes of RUBIK.

Hardware Setup: The experiments were run on a dual-core Intel Core i5 @ 2.6 GHz machine with 3MB of L3 Cache, 8 GB of DDR3 RAM @ 1600 MHz, and 128 GB of PCIe-based flash storage.

Software Setup: RUBIK was implemented in C++ as a single-threaded application.

Setting: We perform a series of tests on the two encoding schemes implemented in RUBIK, and compare them in terms of index sizes, average query execution time and number of hits. Additionally, we vary the number of timeseries and the number of bins we use, in order to compare the changes in speed and performance. For the first set of tests, we used 128 bins in the case of range encoding and 64 intervals for interval encoding, resulting in 127 bins. Queries were run on the whole timeseries as well as smaller segments. The indexes are first loaded into memory before executing any queries so as not to access disk pages during the execution. The data page size used was 96KB, and all caches were cleared before each test.

Test Data: For the evaluation, we tested RUBIK on a real-world neuroscience dataset, consisting of 312,349 timeseries of 1,000 timesteps each. The data comes out of the Blue Brain Project and represents the 6th layer of the neocortical column of a rat. Each timestep of a timeseries contains the voltage value of a particular electrically simulated neuron. As each simulation was performed for 1,000 timesteps, that is the length of each timeseries.

Micro-benchmarks: Three sets of 20 queries each for a total of 60, were performed, with one covering the whole timeseries, the second approximately half, and the third approximately a quarter of the timesteps. The thresholds were chosen randomly from the possible values. Each query was ran 5 times in order to obtain a more accurate, average result.

Approach: We measured the average time required to count the number of results, with the figures showing an overall average query time for all three sets of 20 queries. We then report the percentage of definite hits out of the number of actual results.

6.1 Comparative Analysis

In this section we compare the sizes of the indexes, speed of the queries, and percentage of hits of the range encoding with the interval encoding scheme, using approximately the same number of bins, i.e. $N-1$, where N is the number of bins of range encoding. This is due to the fact that, as described in Section 4, each interval covers $\lfloor N/2 \rfloor - 1$ bins. We use the whole dataset of 312,349 timeseries of 1,000 timesteps, with 128 bins for range encoding and 64 intervals, i.e. 127 bins for interval encoding. We ran the test set of 60 queries, with each one being executed 5 times, and report the average query execution time.

Index Size: Figure 2 shows the sizes of the indexes for range and interval encoding. We can see that using interval encoding results in an index that is approximately 3% larger. Even though the two-dimensional bitmaps are half the size in the observation dimension, as we have only 64 intervals, it does not result in any overall size decrease. However, this is expected, as interval encoding does not compress very well, having shorter consecutive runs of "1s" and "0s".

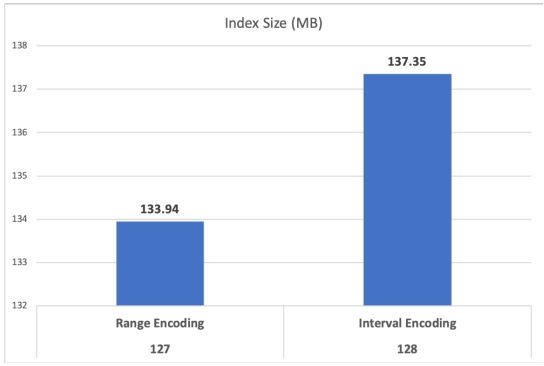


Figure 2: Index Size (MB)

Average Query Execution Time: Figure 3 shows that interval encoding is better suited for both one-sided and two-sided range queries, taking approximately 25% less time to answer both types of queries. This matches what is theoretically argued in [2].

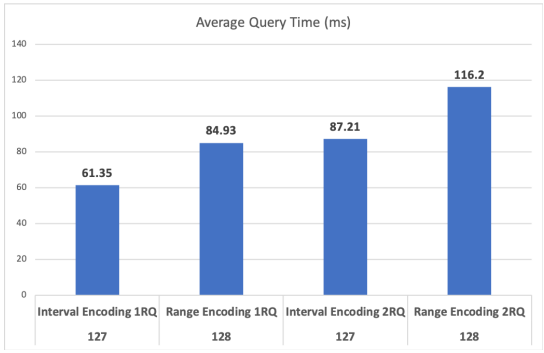


Figure 3: Average Query Execution Time

Percentage of Hits: Figure 4 shows the number of definite hits out of the total number of results, with both types of encoding having very similar performance. As we use the same number of bins and the same binning strategy, this is expected.

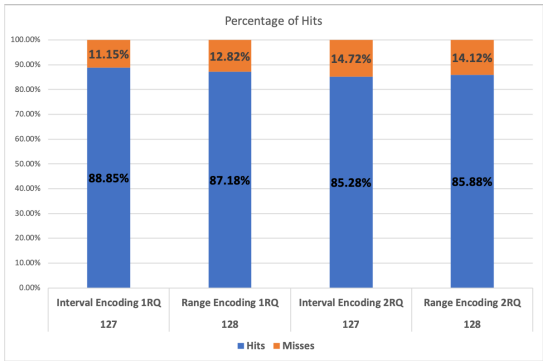


Figure 4: Percentage of Hits

6.2 Scalability Analysis

In this section we analyze how different configurations of number of bins used and size of dataset impacts query execution time and size of the index. When varying the dataset size, we select two subsets of the original data, one 50% smaller (156,174 timeseries) and the other 75% smaller (78,087 timeseries). The same set of 60 queries was run, 5 times each.

6.2.1 *Varying the dataset size.*

Index Size: Figure 5 shows that the size of the index scales linearly with the size of the dataset, and about the same proportion of difference between the range-encoded and interval-encoded index holds. The number of bins used for the test is 128 and 127 for range and interval encoding, respectively.

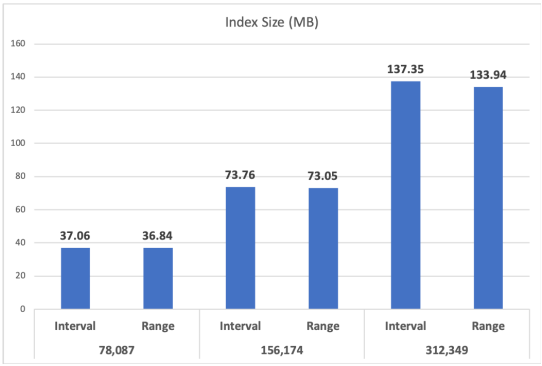


Figure 5: Index Size (MB)

Query Execution Time: Figure 6 shows the average total query execution time for the combined one-sided and two-sided range queries, when varying the size of the dataset. Since we are measuring total time, it is expected that the execution times are shorter when dealing with less data, as the Quadtree nodes we need to perform candidate checks on are smaller. In figure 7 we can see the average total query execution time broken down for one-sided and two-sided queries, with interval encoding showing better performance in all cases.



Figure 6: Query Execution Time

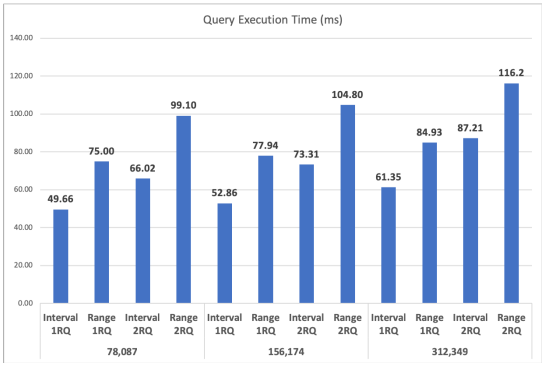


Figure 7: Query Execution Time Broken Down

6.2.2 *Varying the number of bins.*

Index Size: In figure 8 we see that doubling the number of bins, going from 128 bins to 256 bins (127 to 255 in the case of interval encoding), hence the number of bitmaps, results in a 87% increase of the size of the interval-encoded index, and a 135% increase of the range-encoded index. When using more bins, we can see that compression gets better for interval encoding, as we may encounter longer consecutive runs of 1s and 0s, allowing more *ALL*1 and *ALL*0 nodes. This results in a lower index size than for range encoding.

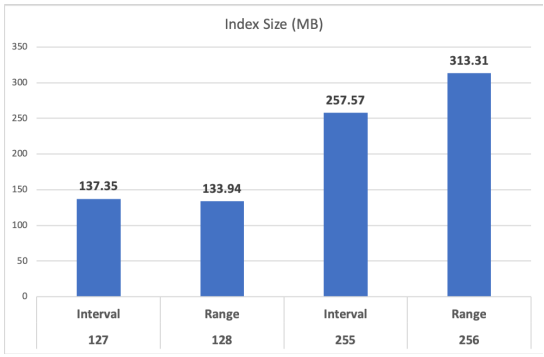


Figure 8: Index Size (MB)

Query Execution Time: Figure 9 shows the average query execution time when varying the number of bins. The average is calculated across both types of queries, i.e. one-sided and two-sided, with interval encoding showing a more significant performance gain when using a larger number of bins. Because we use a larger number of bins however, the query range may intersect more nodes, thus leading to longer query execution times. Figure 10 shows the total average query execution time for one-sided and two-sided queries.

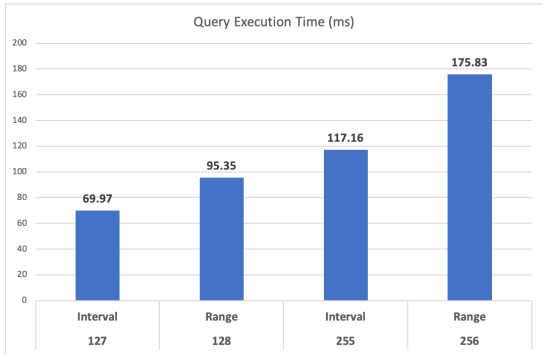


Figure 9: Query Execution Time

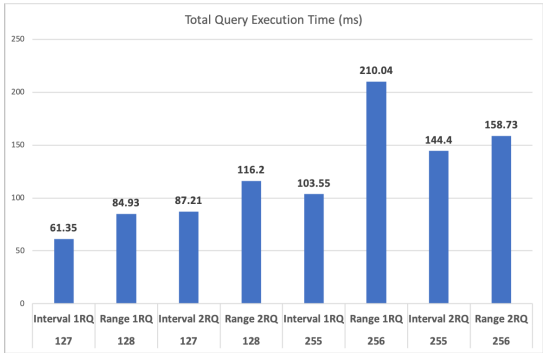


Figure 10: Query Execution Time Broken Down

7 FUTURE WORK

Concerning future extensions to RUBIK, that would be of benefit to its users, we considered, but due to lack of time, have not implemented, various other types of queries and functions. Inspired by the Oracle 8i Database [3], that supports time-series data, we identified functions such as ”TSAvg”, which returns the average value across a timeseries, ”TSMax/TSMin”, which give the maximum/minimum value from a timeseries, ”TSMaxN/TSMinN”, allowing a user-specified K number of top-value elements/lowest-value elements to be returned. The functions could be applied either to the whole timeseries, or easily restricted to a given time-period, by taking advantage of the pruning power of the Quadtree data structure. Besides more queries being implemented, it would also be highly interesting to implement the two-level encoding scheme and perform tests in order to compare its query execution time, index size and number of hits to the results we obtained in this report.

8 CONCLUSION

In this report, we have detailed an extension to RUBIK, an efficient and innovative timeseries indexing approach whereby timeseries data is transformed into two-dimensional bitmaps that are then clustered together in a three dimensional structure, and then decomposed using a Quadtree data structure. We now added the possibility of running two-sided range queries on the range-encoded 2D bitmaps, and implemented interval encoding, with ability to run one-sided and two-sided range queries. We finally compared the two types of encodings in terms of index size, average query execution time and percentage of hits. Based on the empirical analysis presented in this report, we can conclude that interval encoding is better suited for both one-sided and two-sided range queries, while having a comparable index size and percentage of hits.

9 ACKNOWLEDGEMENTS

I would like to thank my project supervisor, Eleni Tzirita Zacharatou. I am profoundly grateful for her supervision and suggestions that guided my research for this project. Her mentorship and support played a key role throughout the development of this project.

REFERENCES

[1] Chee-Yong Chan and Yannis E. Ioannidis. 1998. Bitmap Index Design and Evaluation. *SIGMOD Rec.* 27, 2 (June 1998), 355–366. https://doi.org/10.1145/276305.276336

[2] Chee-Yong Chan and Yannis E. Ioannidis. 1999. An Efficient Bitmap Encoding Scheme for Selection Queries. *SIGMOD Rec.* 28, 2 (June 1999), 215–226. https://doi.org/10.1145/304181.304201

[3] Oracle. [n. d.]. Time Series Functions. Retrieved Jan 10, 2019 from https://docs.oracle.com/cd/F49540_01/DOC/inter.815/a67294/ts_tscon.htm

[4] Rishi Rakesh Sinha and Marianne Winslett. 2007. Multi-resolution Bitmap Indexes for Scientific Data. *ACM Trans. Database Syst.* 32, 3, Article 16 (Aug. 2007). https://doi.org/10.1145/1272743.1272746

[5] Kurt Stockinger. 2001. Design and Implementation of Bitmap Indices for Scientific Data. In *Proceedings of the International Database Engineering &Amp; Applications Symposium (IDEAS '01)*. IEEE Computer Society, Washington, DC, USA, 47–57. http://dl.acm.org/citation.cfm?id=646290.687051

[6] Eleni Tzirita Zacharatou, Thomas Heinis, Farhan Tauheed, and Anastasia Ailamaki. 2015. RUBIK: Efficient Threshold Queries on Massive Time Series. *Proceedings of the 27th International Conference on Scientific and Statistical Database Management* (2015). https://doi.org/10.1145/2791347.2791372 BRAINDB.

[7] Kesheng Wu, Ekow J. Otoo, and Arie Shoshani April. 2004. An Efficient Compression Scheme For Bitmap Indices fk.

[8] Kesheng Wu, Arie Shoshani, and Kurt Stockinger. 2008. Analyses of Multi-level and Multi-component Compressed Bitmap Indexes. *ACM Trans. Database Syst.* 35, 1, Article 2 (Feb. 2008), 52 pages. https://doi.org/10.1145/1670243.1670245

[9] Kesheng Wu, Kurt Stockinger, and Arie Shoshani. 2008. Breaking the Curse of Cardinality on Bitmap Indexes. In *Proceedings of the 20th International Conference on Scientific and Statistical Database Management (SSDBM '08)*. Springer-Verlag, Berlin, Heidelberg, 348–365. https://doi.org/10.1007/978-3-540-69497-7_23